MATERIAL DIDÁCTICO · PMDM

TUTORIAL 02

Inventario con RecyclerView

APRENDER PROGRAMANDO, PASO A PASO

PROGRAMACIÓN MULTIMEDIA Y DISPOSITIVOS MÓVILES · 2.º DAM

TUTORIAL RESUELTO · DATA CLASS · ADAPTER · VIEWHOLDER

Resultado de aprendizaje

Construirás un inventario para el aula: cada material tendrá nombre y número de unidades. Aprenderás a separar modelo, adaptador y pantalla. No necesitarás descargar un proyecto de terceros.

Configuración	Valor
Proyecto	InventarioPMDM
Plantilla	Empty Views Activity · Kotlin y XML
Package	com.example.inventariopmdm
Mínimo del proyecto	El que indique el entorno de clase
Compilación	compileSdk 35 o posterior
Dependencia	androidx.recyclerview:recyclerview:1.4.0

1. Crea el proyecto y añade RecyclerView

1. Crea **InventarioPMDM** con la plantilla Views y ejecútalo una vez en el emulador.
2. En el panel Project abre **Gradle Scripts → build.gradle.kts (Module :app)**. No abras el archivo de nivel de proyecto por equivocación.
3. Busca el bloque **dependencies { ... }**. Añade dentro la línea del recuadro; no sustituyas las dependencias existentes.
4. Comprueba que el **compileSdk** del módulo es 35 o posterior. Si no, instala la plataforma desde **SDK Manager** y ajusta la propiedad al SDK instalado.
5. Pulsa **Sync Now** y espera al fin de la sincronización.

Línea que debes añadir dentro de dependencies

GRADLE / CÓDIGO COMPLETO O FRAGMENTO

```
implementation("androidx.recyclerview:recyclerview:1.4.0")
```

> Si el proyecto usa Groovy (**build.gradle** sin .kts), la equivalencia es: `implementation "androidx.recyclerview:recyclerview:1.4.0"`. No copies simultáneamente ambas versiones.

> **Comprueba:** puedes importar androidx.recyclerview.widget.RecyclerView sin que aparezca como referencia desconocida.

2. Diseña la pantalla y una fila

2.1 activity_main.xml

Este diseño tiene dos entradas, botón, resumen y un RecyclerView que ocupa el espacio restante. Conserva los identificadores exactos.

Archivo: res/layout/activity_main.xml

XML / CÓDIGO COMPLETO O FRAGMENTO

```xml
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="20dp">
    <TextView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Inventario del aula"
        android:textSize="24sp" />
    <EditText
        android:id="@+id/etNombre"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Material"
        android:inputType="textCapSentences" />
    <EditText
        android:id="@+id/etCantidad"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Unidades"
        android:inputType="number" />
    <Button
        android:id="@+id/btnAgregar"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Añadir material" />
    <TextView
        android:id="@+id/tvResumen"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="0 materiales"
        android:textSize="16sp" />
    <androidx.recyclerview.widget.RecyclerView
        android:id="@+id/rvProductos"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1" />
</LinearLayout>
```

2.2 Crea item_producto.xml

1. Haz clic derecho en **res/layout → New → Layout Resource File**.
2. Escribe **item_producto** (sin extensión), elige LinearLayout si te lo solicita y acepta.
3. Sustituye el archivo por este diseño: una fila con dos TextView.

Archivo: res/layout/item_producto.xml

XML / CÓDIGO COMPLETO O FRAGMENTO

```xml
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:padding="12dp">
    <TextView
        android:id="@+id/tvProducto"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Producto"
        android:textSize="18sp" />
    <TextView
        android:id="@+id/tvCantidad"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Cantidad"
        android:textSize="14sp" />
</LinearLayout>
```

> **Comprueba:** existen dos layouts: activity_main.xml para la pantalla e item_producto.xml para una fila.

3. Crea el modelo Producto

Haz clic derecho en tu paquete Kotlin → **New → Kotlin Class/File** → File: **Producto**. Pega la data class siguiente. Cada objeto guarda datos de un único material.

Archivo: Producto.kt

KOTLIN / CÓDIGO COMPLETO O FRAGMENTO

```kotlin
package com.example.inventariopmdm

data class Producto(
    val nombre: String,
    val cantidad: Int
)
```

val hace que las propiedades no puedan reasignarse; si necesitas cambiar una cantidad, puedes crear otro Producto con **copy(cantidad = nuevoValor)**.

4. Crea el adaptador

Crea **ProductoAdapter.kt** en el mismo paquete. El ViewHolder identifica los dos TextView de cada fila. **onCreateViewHolder** infla el XML y **onBindViewHolder** coloca el producto correcto en esa fila.

Archivo: ProductoAdapter.kt

KOTLIN / CÓDIGO COMPLETO O FRAGMENTO

```
package com.example.inventariopmdm

import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.widget.TextView
import androidx.recyclerview.widget.RecyclerView

class ProductoAdapter(private val datos: List<Producto>) :
    RecyclerView.Adapter<ProductoAdapter.ProductoViewHolder>() {

    class ProductoViewHolder(vista: View) :
        RecyclerView.ViewHolder(vista) {
        val nombre: TextView = vista.findViewById(R.id.tvProducto)
        val cantidad: TextView = vista.findViewById(R.id.tvCantidad)
    }

    override fun onCreateViewHolder(
        parent: ViewGroup, viewType: Int
    ): ProductoViewHolder {
        val vista = LayoutInflater.from(parent.context)
            .inflate(R.layout.item_producto, parent, false)
        return ProductoViewHolder(vista)
    }

    override fun onBindViewHolder(
        holder: ProductoViewHolder, position: Int
    ) {
        val producto = datos[position]
        holder.nombre.text = producto.nombre
        holder.cantidad.text = "Unidades: ${producto.cantidad}"
    }

    override fun getItemCount(): Int = datos.size
}
```

Qué hace cada función

- **getItemCount:** devuelve cuántas filas hay en la colección.
- **onCreateViewHolder:** crea una nueva vista solo cuando hace falta.
- **onBindViewHolder:** lee datos[position] y actualiza los controles de una fila.
- **ViewHolder:** conserva las referencias a los TextView para reutilizarlas.

5. Conecta el formulario con RecyclerView

Abre **MainActivity.kt**, sustituye la clase por este código y conserva el package correcto. Observa que los cálculos del resumen están en una función independiente, mientras la Activity maneja los controles.

Archivo: MainActivity.kt

KOTLIN / CÓDIGO COMPLETO O FRAGMENTO

```
package com.example.inventariopmdm

import android.os.Bundle
import android.widget.Button
import android.widget.EditText
import android.widget.TextView
import androidx.appcompat.app.AppCompatActivity
import androidx.recyclerview.widget.LinearLayoutManager
import androidx.recyclerview.widget.RecyclerView

class MainActivity : AppCompatActivity() {
    private val productos = mutableListOf<Producto>()

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val entradaNombre = findViewById<EditText>(R.id.etNombre)
        val entradaCantidad = findViewById<EditText>(R.id.etCantidad)
        val boton = findViewById<Button>(R.id.btnAgregar)
        val resumen = findViewById<TextView>(R.id.tvResumen)
        val lista = findViewById<RecyclerView>(R.id.rvProductos)

        val adaptador = ProductoAdapter(productos)
        lista.layoutManager = LinearLayoutManager(this)
        lista.adapter = adaptador

        boton.setOnClickListener {
            val nombre = entradaNombre.text.toString().trim()
            val cantidad = entradaCantidad.text.toString().toIntOrNull()
            if (nombre.isEmpty() || cantidad == null || cantidad < 1) {
                resumen.text = "Introduce un material y unidades positivas"
            } else {
                productos.add(Producto(nombre, cantidad))
                adaptador.notifyItemInserted(productos.lastIndex)
                resumen.text = resumenInventario(productos)
                entradaNombre.text.clear()
                entradaCantidad.text.clear()
            }
        }
    }

    private fun resumenInventario(datos: List<Producto>): String {
        val unidades = datos.sumOf { it.cantidad }
        return "Referencias: ${datos.size} · Unidades: $unidades"
    }
}
```

Flujo de ejecución

1. La lista empieza vacía; **ProductoAdapter(productos)** recibe esa misma lista.
2. **LinearLayoutManager(this)** dispone las filas verticalmente.
3. Al pulsar el botón se convierten unidades con **toIntOrNull()** y se exige un valor positivo.
4. Si el dato es válido se crea **Producto(nombre,cantidad)**, se añade y se llama a **notifyItemInserted**.
5. La función **resumenInventario** calcula el total de unidades mediante **sumOf**.

6. Pruebas y errores habituales

Entrada	Comportamiento
Cuaderno / 4	Fila Cuaderno, Unidades: 4; resumen 1 / 4
Lápiz / 6	Dos filas; resumen referencias 2 / unidades 10
Nombre vacío / 2	Mensaje de error, sin nuevas filas
Folio / 0	Mensaje de error
Folio / letras	No se añade; no se cierra la app
20 materiales	Desplazamiento vertical normal

> **Si aparece una pantalla vacía:** verifica el atributo layout_weight de RecyclerView, que has asignado un LayoutManager y que el adaptador apunta a la misma lista que modificas.

Ampliaciones para después del tutorial

- Añade el precio unitario (Double) al modelo y muestra el total de cada fila.
- Evita nombres repetidos y muestra un aviso útil.
- Incorpora un clic sobre una fila para mostrar su ficha en un TextView independiente.

Consulta oficial: Crear listas dinámicas Android Views.
https://developer.android.com/develop/ui/views/layout/recyclerview

> **Entrega:** código completo, captura con tres materiales y captura de error. A continuación, **UD2 · Relación 2**.